Project 2, FYS-STK3155
When is a Neural Network worth it? Regression of the Runge Function and classification of Fashion-MNIST

Albert Sjåvåg,[1] Teodor Aursnes,[1] and Jørgen S. Florholmen[1]

[1] *University of Oslo, Department of Physics*

(Dated: November 9, 2026)

Neural networks can model non-linear problems, but they cost more to tune than simple linear models. We investigate when this extra complexity is worth it. We implemented a fully connected feed-forward neural network with backpropagation. The network was compared with OLS, Ridge and Lasso on fitting Runge data. For classification we compared the network with logistic regression and Scikit-Learn on the Fashion-MNIST data. The gradients were checked against JAX. We chose Adam for its stability, and cross-validation on the Runge data due to its small size, but not on Fashion-MNIST due to its large size. The network has a slightly higher test MSE of 0.0065 compared to OLS 0.0061. The network beats logistic regression with test accuracy 0.875 against 0.844, and it comes close to Scikit-Learn's 0.883. We found that the extra complexity is worth it for high-dimensional, structured data like images, but less so for simple one-dimensional problems.

I. INTRODUCTION

Today we find neural networks everywhere, in image recognition, speech recognition, language translation and self-driving cars [1]. The universal approximation theorem says that a network with one hidden layer can approximate any continuous function [2], but it does not say anything about how easy it is to find the weights. The weights are found by training, and the result depends on choices such as the learning rate, the optimizer and the architecture. Thus, choosing the right hyperparameters becomes important.

Polynomials are a natural first choice for approximating a function, e.g. the Runge function $f(x) = 1/(1 + 25x^2)$ [3]. In Project 1 [4] we fitted this function with polynomials using ordinary least squares (OLS), Ridge regression [5] and Lasso regression [6]. This required us to choose the features, the powers of x ourselves. Also, the polynomials of high degree oscillate near the end points. A neural network instead learns its own features from the data [1]. It is then relevant to ask whether a feed-forward neural network does a better job than these linear models, and when the extra complexity is worth it.

In this study we implement our own feed-forward neural network with backpropagation and verify its gradients against automatic differentiation in JAX [7]. We first train the net on data generated from the Runge function and compare it with OLS, Ridge and Lasso. We study how the optimizer, the activation function, the architecture and L_1 and L_2 penalties affect the error. We then train the network for classification on the Fashion-MNIST images [8], and compare it with logistic regression and Scikit-Learn [9]. The two problems are chosen to be different, the first is small, smooth and one-dimensional, while the second consists of 70 000 images with 784 pixels each. As we will see, increasing model complexity is not always the best way to attack a problem.

The rest of this paper is structured as follows. Section II presents the theory and methods used for building the network. In Section III we go through in detail how we implemented the network, what kind of data we have worked with and how we implemented our model selection and training. Our findings and discussion are presented in Section IV. Finally, in Section V we provide a conclusion to our discussion.

All code is available at https://github.com/albertsjavag/Project2_FysStk

II. THEORY & METHODS

A. Neural networks

The core objective of machine learning is to make predictions based on patterns from data. In Project 1, we used OLS, Ridge and Lasso, which are linear in their parameters and rely on manually chosen features, such as the powers of x in the design matrix. For more complex inputs, such as the 784 pixels in a Fashion-MNIST image, choosing suitable features and their interactions becomes almost impossible. Neural networks address this by learning features during training through a sequence of weighted sums and nonlinear activation functions. This section introduces these building blocks, defines the cost functions and regularization penalties, and derives the gradients needed to train the network using backpropagation.

1. *Feed-forward neural network*

A feed-forward neural network maps an input to an output through a sequence of layers: an input layer, one or more hidden layers, and an output layer. Information moves forward through the network, with each layer using the output of the previous layer as its input. For the problems considered here, the input is either a single

value x for the Runge function or a vector of 784 pixel values for a Fashion-MNIST image.

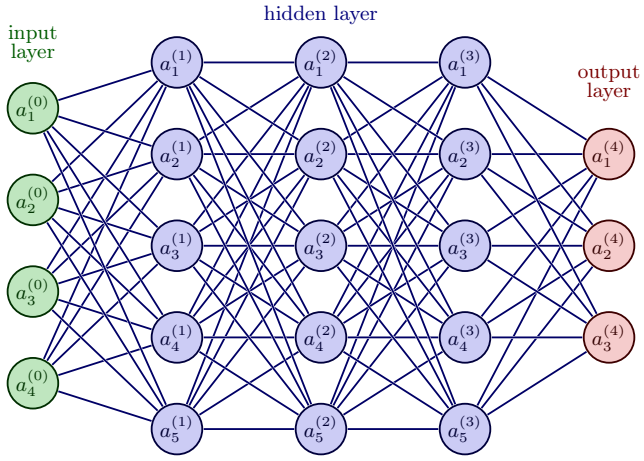


FIG. 1. An example of a fully connected feed-forward neural network with four input nodes, three hidden layers of five nodes each, and three output nodes. The layer widths are $(M_0, M_1, M_2, M_3, M_4) = (4, 5, 5, 5, 3)$. Nodes represent activations, and connections represent trainable weights. Figure from Neutelings [10].

We number the layers $l = 0, \ldots, L$, where layer 0 is the input layer and layer L is the output layer. Let M_l denote the number of nodes in layer l. Fig. 1 illustrates a fully connected feed-forward neural network with an input layer, three hidden layers and an output layer, giving $L = 4$. Each node is connected to every node in the following layer. Let $\boldsymbol{a}^0 = \boldsymbol{x}$ denote the input, represented as a row vector. In a fully connected layer l, each node forms a weighted sum of the inputs and adds a bias. Collecting these calculations in vector form gives

$$\boldsymbol{z}^l = \boldsymbol{a}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = f^l\left(\boldsymbol{z}^l\right),$$

where $\boldsymbol{W}^l \in \mathbb{R}^{M_{l-1}\times M_l}$ is the weight matrix, $\boldsymbol{b}^l \in \mathbb{R}^{1\times M_l}$ is the bias vector, and f^l is the activation function. In the hidden layers, the activation function is applied elementwise. The weights and biases are learned during training, and the prediction is given by $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$.

The weighted sum and bias form an affine transformation. If every activation function were the identity, composing these layers would produce another affine transformation, regardless of the number of layers. Nonlinear activation functions allow the hidden layers to learn more complex features and relationships. We therefore need to consider which activation functions to use and how their properties affect training.

2. *Activation functions*

Activation functions introduce nonlinearity, but their derivatives also affect how the network learns. During backpropagation, the gradient passes through the weight matrices and activation derivatives of successive layers. Repeated multiplication by small derivatives can therefore contribute to vanishing gradients, slowing learning in earlier layers.

The sigmoid function is defined as

$$\sigma(z) = \frac{1}{1+e^{-z}}.$$

The sigmoid maps real-valued inputs to the interval $(0, 1)$, approaching 0 as $z \to -\infty$ and 1 as $z \to \infty$. Its output can therefore be interpreted as a probability. Applying the chain rule gives

$$\sigma'(z) = \sigma(z)\big[1 - \sigma(z)\big]. \tag{1}$$

(derived in Appendix A 1) The derivative reaches its maximum of $1/4$ at $z = 0$ and approaches zero for large $|z|$. In these saturated regions, sigmoid nodes transmit very small gradients, which can make deep networks difficult to train.

The rectified linear unit (ReLU) and its derivative are

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

For positive inputs, ReLU does not shrink the gradient locally. For negative inputs, however, its derivative is zero. A node that remains inactive for all training samples receives zero gradients for its incoming weights and bias, potentially becoming a "dead" ReLU.

Leaky ReLU introduces a small positive slope α for negative inputs:

$$\mathrm{LReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \le 0, \end{cases} \qquad \mathrm{LReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z < 0. \end{cases}$$

We use $\alpha = 0.01$, allowing a gradient signal for negative inputs and reducing the risk of dead nodes. Neither ReLU nor Leaky ReLU is differentiable at $z = 0$; for gradient computations, we assign values of 0 and α, respectively.

The output activation must match the prediction task. For regression, we use the identity function, allowing unrestricted real-valued predictions. For Fashion-MNIST classification, we need an output that represents probabilities for the different classes. We introduce the softmax function for this purpose in the next subsection, together with the cost functions used to measure prediction errors.

3. *Cost functions*

A cost function measures the discrepancy between the predictions $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$ and the targets. Training minimizes this cost over the weights and biases. The losses considered here have a maximum-likelihood interpretation. Assuming independent observations, Gaussian targets with

fixed variance gives MSE up to a positive scaling factor and an additive constant. Bernoulli and categorical targets give binary and multiclass cross-entropy, respectively.

For scalar regression, the mean-squared error is

$$C_{\mathrm{MSE}} = \frac{1}{n}\sum_{i=1}^{n}\left(a_i^L - y_i\right)^2, \qquad \frac{\partial C_{\mathrm{MSE}}}{\partial a_i^L} = \frac{2}{n}\left(a_i^L - y_i\right).$$

Here, n is the number of observations, y_i is the target for observation i, and a_i^L is its predicted value. With the identity output activation, $a_i^L = z_i^L$, so the derivative with respect to z_i^L is the same.

For binary classification, $y_i \in \{0, 1\}$ and a sigmoid output represents the predicted probability $a_i^L = P(y_i = 1 \mid \boldsymbol{x}_i)$. The binary cross-entropy is

$$C_{\mathrm{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log a_i^L + (1 - y_i)\log\left(1 - a_i^L\right)\right]. \quad (2)$$

For each observation, only the term corresponding to its target class contributes. Using the sigmoid derivative from Equation (1) and the chain rule (Appendix A 2), the gradient with respect to the output score is

$$\frac{\partial C_{\mathrm{BCE}}}{\partial z_i^L} = \frac{1}{n}\left(a_i^L - y_i\right).$$

For multiclass classification, the output layer contains K nodes, one for each class $k = 0, \ldots, K-1$. Softmax converts the output scores into class probabilities:

$$a_{ik}^L = \frac{\exp\left(z_{ik}^L\right)}{\sum_{m=0}^{K-1}\exp\left(z_{im}^L\right)}. \quad (3)$$

These probabilities are positive and sum to one for each observation. For numerical stability, softmax can be evaluated after subtracting $\max_m z_{im}^L$ from every score; this leaves the probabilities unchanged.

Using one-hot targets, where $y_{ik} = 1$ for the correct class and 0 otherwise, the multiclass cross-entropy is

$$C_{\mathrm{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log a_{ik}^L.$$

The inner sum retains only the negative logarithm of the probability assigned to the correct class.

Combining the softmax derivative with the chain rule and using $\sum_k y_{ik} = 1$ (Appendix A 3), the gradient with respect to the output scores is

$$\frac{\partial C_{\mathrm{CE}}}{\partial z_{ij}^L} = \frac{1}{n}\left(a_{ij}^L - y_{ij}\right). \quad (4)$$

All three matched pairs produce an output error proportional to the difference between prediction and target:

$$\boldsymbol{\delta}_i^L = \frac{\partial C}{\partial \boldsymbol{z}_i^L} \propto \boldsymbol{a}_i^L - \boldsymbol{y}_i. \quad (5)$$

The proportionality factor is $2/n$ for MSE and $1/n$ for the cross-entropy losses. Using sigmoid with MSE instead gives

$$\frac{\partial C_{\mathrm{MSE}}}{\partial z_i^L} = \frac{2}{n}\left(a_i^L - y_i\right)a_i^L\left(1 - a_i^L\right).$$

The additional sigmoid factor can make the gradient very small when the output saturates, even for confidently incorrect predictions. The choice of output activation and cost function therefore matters for training. The output error is the starting point for backpropagation.

4. *Regularization*

A flexible network can fit noise in the training data. Regularization discourages this by penalizing the weights. Writing $\mathcal{W}$ for the collection of weight matrices, the regularized cost is

$$C_\lambda = C + \lambda\Omega(\mathcal{W}), \qquad \lambda \geq 0,$$

where λ controls the penalty strength. The two penalties considered are

$$\Omega_{L_2}(\mathcal{W}) = \sum_{l=1}^{L}\left\|\boldsymbol{W}^l\right\|_F^2 = \sum_{l=1}^{L}\sum_{p,q}\left(w_{pq}^l\right)^2,$$

$$\Omega_{L_1}(\mathcal{W}) = \sum_{l=1}^{L}\sum_{p,q}\left|w_{pq}^l\right|.$$

Here, $\|\cdot\|_F$ is the Frobenius norm. As in Ridge and Lasso regression, L_2 penalizes large weights, while L_1 encourages sparsity. Either penalty can be added to any of the cost functions above; setting $\lambda = 0$ recovers the unregularized cost.

The resulting weight gradients are

$$\frac{\partial C_\lambda}{\partial \boldsymbol{W}^l} = \frac{\partial C}{\partial \boldsymbol{W}^l} + \begin{cases} 2\lambda\boldsymbol{W}^l, & L_2, \\ \lambda\ \mathrm{sign}\left(\boldsymbol{W}^l\right), & L_1. \end{cases} \quad (6)$$

For L_1, this denotes a subgradient, with the elementwise choice $\mathrm{sign}(0) = 0$. We leave biases unpenalized, allowing activation thresholds and output offsets to adjust without a penalty towards zero.

For a given forward pass, the penalty contributes directly to the weight gradients without changing the output error $\boldsymbol{\delta}^L$ or its backward recursion. With the cost fully specified, we now need an efficient way to compute its gradients with respect to every weight and bias. This is provided by backpropagation.

5. *Backpropagation*

We define the error signal at layer l as $\boldsymbol{\delta}^l = \partial C/\partial \boldsymbol{z}^l$ and start with $\boldsymbol{\delta}^L$ from Equation (5). By the chain rule, the

signal passes backwards through the transposed weights and is scaled by the local activation derivative:

$$\boldsymbol{\delta}^l = \left(\boldsymbol{\delta}^{l+1}\left(\boldsymbol{W}^{l+1}\right)^T\right) \odot f^{l\prime}\left(\boldsymbol{z}^l\right), \qquad l = L-1, \ldots, 1,$$

where $\odot$ denotes elementwise multiplication. With $\boldsymbol{a}^{l-1}$ and $\boldsymbol{\delta}^l$ arranged as batch matrices with one row per observation, the gradients are

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \left(\boldsymbol{a}^{l-1}\right)^T \boldsymbol{\delta}^l, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \sum_{i=1}^{n} \boldsymbol{\delta}_i^l.$$

We add the penalty term from Equation (6) to the weight gradient. Reusing the error signals gives gradients for all weights and biases in one backward pass, at a computational cost comparable to one forward pass.

III. IMPLEMENTATION

A. Network implementation and verification

We built our own neural network code in Python with NumPy, using object-oriented programming. The network class stores the weights and biases for each layer with a forward pass, backpropagation and training. The repository has shared modules plus one script per analysis. We made a Jupyter notebook to run everything once for the reader to verify our code and output.

The weights were initialized from a normal distribution scaled by the number of inputs, with He initialization for ReLU and Leaky ReLU. The biases started at 0.01. For softmax and cross-entropy, the output error was computed directly from Eq. (5). The penalties L_2 and L_1 were added to the weight gradients as in Eq. (6). The network was trained with stochastic gradient descent on mini-batches with the learning rate adapted by RMSprop or Adam [11, Secs. 4.7, 4.10–4.11], while plain full batch gradient descent was used for comparison. These implementations are only useful once verified.

The backpropagation gradients were compared with automatic differentiation in JAX [7] with MSE, L_2 penalized MSE and softmax with cross-entropy. We checked the shapes of the layers, and that the cost decreased during training, and we implemented unit tests with pytest. For our final results we did an external check, comparing our results to Scikit-Learn's [9] logistic regression and MLP with the same settings (Sec. IV C 2).

B. Data sets

We generated Runge data with $n = 100$ evenly spaced data points in the interval $[-1, 1]$ with Gaussian noise of $\sigma = 0.1$. The data were split into 80% training data and 20% test data, and then we used 20% of the training data for validation. Our benchmark is OLS, Ridge and Lasso of degree 12 with penalties found in Project 1 [4]. The columns of the design matrix were standardized with the mean and standard deviation of the training data, also the targets were centered so no intercept column is needed. The Fashion-MNIST data set consists of 60 000 training images and 10 000 test images of 28×28 pixels flattened to 784 features scaled to $[0, 1]$ [8]. Figure 2 shows examples of the 10 classes we want to predict. The test set was only used once, for the final evaluation, so we used an 80/20 split on the 60 000 training images, where the 20% is our validation data used to choose hyperparameters.



FIG. 2. One example image from each of the ten Fashion-MNIST classes.

C. Model selection and training

For the regression, activation and architecture were compared on the validation split. We chose the learning rate and penalty with a 5-fold cross-validation with scaling fitted inside each fold, with standard errors calculated by $s/\sqrt{K}$. Based on Sec. IV A, we used Adam [12] with $\gamma = 0.01$, 1000 epochs and batch size of 16, except for the penalty study where γ was chosen by cross-validation together with λ.

In the classification, we used a softmax output with cross-entropy (Sec. II A 3). We searched γ and λ with a fixed network of one hidden layer of 100 sigmoid nodes. Architectures were then compared using these fixed values. For classification we used the L_2 penalty, since L_1 and L_2 gave the same CV error on the Runge data (Sec. IV B 2). Furthermore, we used Adam with 10 epochs and batch size 100. These were not tuned because of run time, which is a limitation of this study.

The metric used for regression is the standard MSE. For classification we measured prediction accuracy by simply averaging the successful ones: Accuracy $= (1/n)\sum_i \mathbf{1}_{\{\tilde{y}_i = y_i\}}$. We also used a confusion matrix, per-class precision, recall and F_1 defined with TP, FP and FN in [11, Sec. 5.12]. The training cost we monitored is the mean cross-entropy on the training set after each epoch, without the penalty term.

The results of these searches are presented in Sec. IV.

IV. RESULTS AND DISCUSSION

A. Regression: neural network versus OLS

Before we compare our network with OLS we should figure out which optimization method looks promising. Therefore, in Figure 3 we plot validation MSE against $\gamma \in \{10^{-4}, ..., 10^{0}\}$ for the network using the sigmoid as activation with one layer of 50 nodes. For plain gradient descent our best learning rate ($\gamma = 0.1$) gave a validation error of 0.0896. For RMSprop ($\gamma = 0.01$) we get 0.014. Lastly, we get the lowest error using Adam with $\gamma = 0.1$, which gives an error of 0.0135.

For further discussion, we have chosen to use Adam. Not primarily because it has the lowest error, but because it seems to be the most stable at different levels of γ, having a wider flat area. Within this area, the error lies between 0.0135 and 0.0149 for γ from 0.003 to 0.1. We therefore used $\gamma = 0.01$ in the middle of the area, rather than $\gamma = 0.1$ near its edge. But this comparison has a limitation: we can clearly see that Adam is better than plain GD, which stays flat at ~ 0.17 from $\gamma = 10^{-4}$ to $\gamma = 10^{-1.5}$, but we cannot be sure *why*. The reason is that both Adam and RMSprop are using minibatches of size 16, whereas plain GD uses full batch. Therefore, we cannot decide whether the difference comes from the lack of adaptive steps or the fact that the plain GD takes four times fewer updates than RMSprop and Adam.

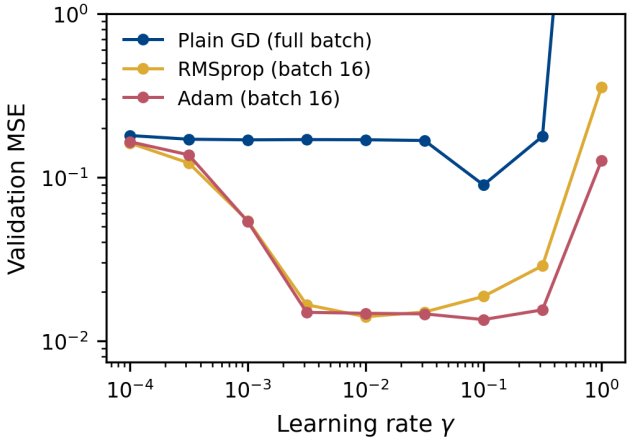


FIG. 3. Validation MSE against the learning rate γ for the network [50] (sigmoid) after 1000 epochs, with plain GD (full batch), RMSprop and Adam (minibatches of 16). Lowest error: Adam 0.0135 ($\gamma = 10^{-1}$). Both axes are logarithmic.

In Figure 4(a) we fit $n = 100$ data points from the Runge function [3]. The blue graph shows our OLS fit of degree 12 which turns out to have the lowest prediction error as seen in Table I. The red curve shows our neural network fit with 5 nodes, and in panel (b) we have the decomposition of the 5 contributions to the fit. In Table I we compare errors when using different amounts of layers and nodes, and as we can see OLS of degree 12 has a lower error than our neural network with 10401 parameters. Since our data is generated from the true Runge function with known noise, the lower bound on the expected test MSE is known, being $\sigma^2 = 0.01$ in this case. Notice that all errors are at or below this lower bound. This is most likely because they share the same noise on the 20 test data points and that the noise happened to be small for our test set. With only 20 data points each MSE is also uncertain. If we assume independent, approximately normally distributed test errors, we can compute the standard deviation. For our values we get SD $\approx 0.32 \cdot$ MSE, i.e. 0.002–0.003 which is about as large as the differences between the models in Table I (see Appendix A 4 to see how we came up with those numbers). We also notice that there is no obvious relationship between the number of parameters and the error. Therefore we are limited to say that we cannot conclude on which model approximates the best or whether more parameters help from these results alone.

TABLE I. Test MSE on the Runge data ($n = 100$, $\sigma = 0.1$, 80/20 split) for OLS with polynomial degree 12 (best model from project 1) and for networks with sigmoid hidden layers and a linear output, trained with Adam ($\gamma = 0.01$, 1000 epochs, batch size 16).

Model	Parameters	Test MSE
OLS, degree 12	12	**0.0061**
NN [5]	16	0.0076
NN [50]	151	0.0087
NN [100]	301	0.0100
NN [50, 50]	2 701	0.0082
NN [100, 100]	10 401	0.0065

Although the errors alone cannot rank the models, we can use Figure 4 to visualize the actual differences between a neural network and the OLS model. The first thing we notice is the well-known Runge phenomenon in the OLS tails [3]. As discussed in Project 1 [4], for equally spaced points, the error grows near $x = \pm 1$ as we increase the polynomial degree. On the other hand, we see that the OLS is more accurate than the neural network near the top, where the network overshoots to about 1.15. The network seems to follow the noisy data points, which lie above the true Runge function. But in contrast to OLS, the five-node network has smooth tails and follows the Runge function closely. So OLS loses in the tails whereas the neural network loses near the top. The errors lie in different places and weigh about the same in total. This could be why the MSE values are almost the same, and why the table alone cannot separate them.

Panel (b) of Figure 4 shows each node's contribution $v_j\sigma(w_j x + b_j)$, which, together with the output bias, sum to the red fit; two steep steps of opposite sign near $x = 0$ form the peak, while the three slower terms shape the tails. The network thus learns its own features from the data, in contrast to the fixed powers $x, \ldots, x^{12}$ in OLS.

The universal approximation theorem tells us that a good approximation exists [11, p. 319], and our network seems to have found one with only five nodes.

The main limitation in these results is that they are all based on a single seed and training–test split, with both small validation and test sets. This makes all comparisons uncertain, including the choice of optimizer, the comparison between architectures, and the comparison with OLS. Several seeds or k-fold cross-validation, as in Project 1 [4], would give more reliable results. Furthermore, γ was tuned only for [50] and could be tuned separately for each architecture. Finally, comparing Adam with plain SGD using minibatches of 16 and otherwise identical training settings would isolate the effect of the update rule.

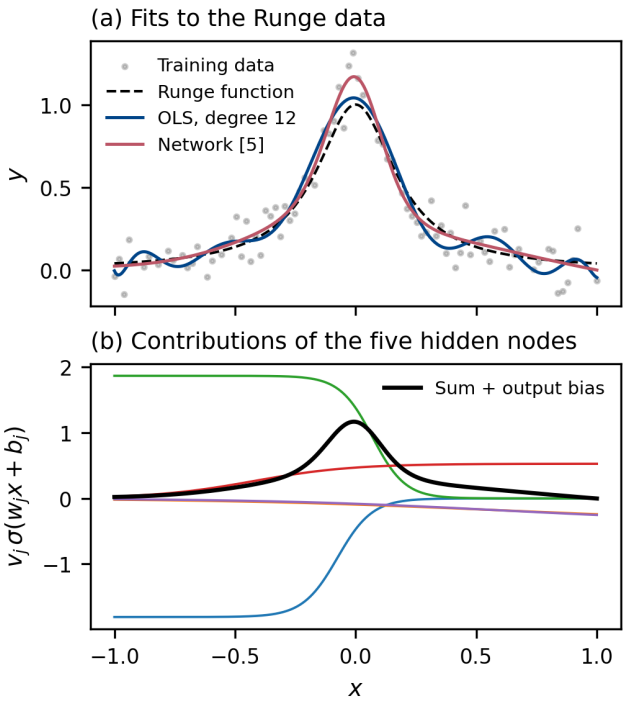


FIG. 4. (a): OLS of degree 12 and a network with one hidden layer of five sigmoid nodes, fitted to the Runge data. (b): the network output written as the sum of the five hidden-node terms plus the output bias.

B. Tuning the network

1. *Activation functions and architecture*

In Section IV A we saw that the network with 50 or 100 nodes and 1 or 2 layers did not outperform OLS in prediction error when using sigmoid as activation. In Figure 5 we explore validation MSE on networks with 1 to 4 layers and with 5, 10, 25, 50 and 100 nodes per layer with sigmoid, ReLU and Leaky ReLU activations. The results appear in Table II, where we see ReLU ([100]) performs best with a validation MSE of 0.0098. With that said Leaky ReLU ([100]) has almost the same result with an MSE of 0.0099. Sigmoid with [25, 25] has an error of 0.0114.

Further, in the context of overfitting data we can take a look at the gap between training MSE and validation MSE. A network with training MSE well below $\sigma^2 = 0.01$, and higher validation MSE is probably overfitting, whereas a network with both high training MSE and validation MSE is probably underfitting. The extra rows for ReLU and Leaky ReLU in Table II illustrate this. ReLU with [25, 25, 25] has a low training MSE and a gap between training and validation MSE of 0.0203, and Leaky ReLU [100, 100] has a low training MSE of 0.0067 and a gap of 0.0224, both signaling overfitting. For Sigmoid specifically, the worst architecture is [100]×4 with low training MSE of 0.0078 and gap 0.0109, mildly overfitting compared with ReLU. Additionally, both ReLU and Leaky ReLU activations show signs of underfitting in their worst architecture, [5], with a relatively high training MSE of 0.0223, for both, and a corresponding high validation MSE of 0.0524 and 0.0525, respectively.

TABLE II. Training and validation MSE for the best and the worst architecture (lowest and highest validation MSE among networks of 1–4 hidden layers with 5–100 nodes) for each hidden activation. Adam, $\gamma = 0.01$, 1000 epochs. For ReLU and Leaky ReLU the worst network, [5], underfits; the extra row is a large network with low training MSE and high validation MSE.

Activation	Architecture	Train MSE	Val. MSE
Sigmoid	best: [25, 25]	0.0086	**0.0114**
	worst: [100] × 4	0.0078	0.0187
ReLU	best: [100]	0.0084	**0.0098**
	worst: [5]	0.0223	0.0524
	[25, 25, 25]	0.0062	0.0265
Leaky ReLU	best: [100]	0.0086	**0.0099**
	worst: [5]	0.0223	0.0525
	[100, 100]	0.0067	0.0291

But why look at the models that are not performing the best? Why not just choose the model with lowest validation MSE? If we do look at the best validation MSE for all three activation functions, we see that they are all very close to $\sigma^2 = 0.01$, and almost identical, with a difference between Sigmoid (worst of the three) and ReLU (best of the three) of only 0.0016. As in Section IV A, we can take a look at the standard deviation of the MSE. In this case, with 16 validation points, we get SD $\approx 0.35 \cdot$ MSE, i.e. 0.0035–0.0040, which is more than twice as large as the difference between the three activation functions (derivation in Appendix A 4). Hence, we cannot choose any activation based on these results alone.

A more interesting criterion is how stable the activations are across different architectures. The span between best and worst validation MSE (within the same

activation) is a simple measure of stability: For Sigmoid worst − best = 0.0187 − 0.0114 = 0.0073, and for ReLU and Leaky ReLU the span is 0.0426, which is approximately six times bigger. The stability is also very well illustrated in Figure 5. We see that Sigmoid has darker colors all over, whereas both ReLU and Leaky ReLU tend to show some brighter colors. A likely reason is that Sigmoid is a smooth function, unlike both ReLU and Leaky ReLU which are piecewise linear. A few Sigmoid nodes can fit the smooth peak of the Runge function, while a small ReLU or Leaky ReLU network has too few straight line segments to fit it well (underfitting). A larger ReLU or Leaky ReLU network can add enough bends to follow the noise (overfitting), while Sigmoid nodes would need to learn much steeper curves to do the same. Therefore, for the rest of this section, we choose to continue with the best-performing architecture, from the most stable activation function. That is Sigmoid [25,25], with a small gap of 0.0028 between the training and validation MSE suggesting that the network barely overfits.

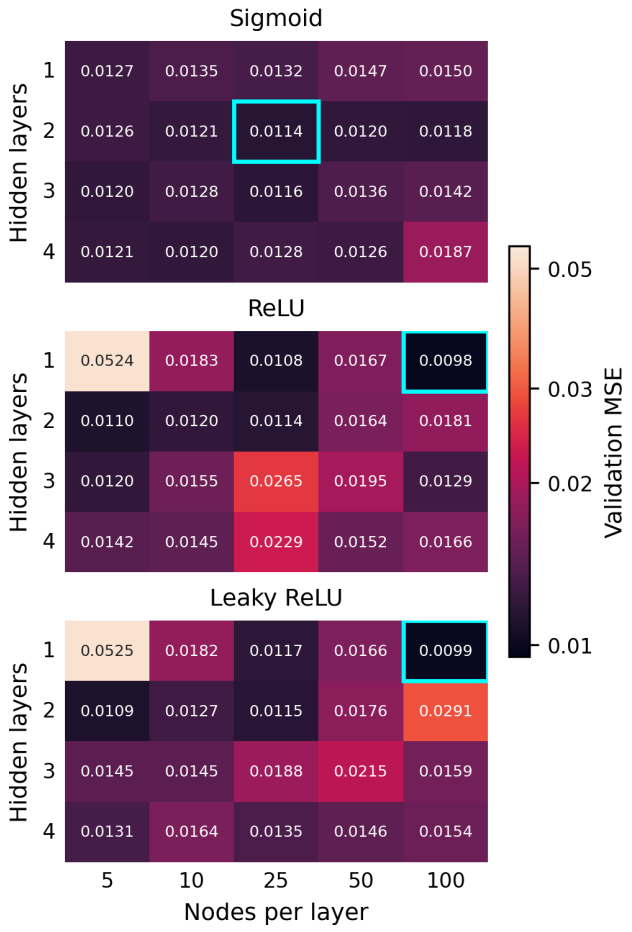


FIG. 5. Validation MSE as a function of the number of hidden layers and nodes per layer for sigmoid, ReLU and Leaky ReLU hidden layers (Adam, $\gamma = 0.01$, 1000 epochs). Logarithmic color scale. Coloured box: lowest MSE for each activation

2. *Regularization*

We next investigate whether regularization improves prediction on unseen data for the sigmoid network [25, 25] selected in Section IV B 1. In Figure 6 we explore CV-MSE for L_2 and L_1 penalties using our structure so far. We used five-fold cross-validation MSE for different learning rates γ and penalties λ. The white boxes mark the combinations where CV MSE lies within one standard error of the minimum, $\mathrm{CV}(\gamma, \lambda) \leq \mathrm{CV}_{\min} + \mathrm{SE}_{\min}$ [11, Eq. (3.96)], and colored box marks the minimum. Both penalties reach their minimum at the same $\gamma = 3 \times 10^{-3}$ and $\lambda = 10^{-5}$, with CV MSE of 0.0111 ± 0.0010. The unpenalised network where $\lambda = 0$ also lies inside the white boxes for both norms, so the penalty gives no measurable improvement. This is consistent with the small training–validation MSE gap of 0.0028 found in Section IV B 1, which suggests limited overfitting for regularization to reduce. For $\lambda \geq 10^{-3}$, the CV MSE rises to 0.04–0.11, indicating underfitting. However, since the selected network shows little overfitting, an overfitting architecture such as ReLU [25, 25, 25] would provide a more informative test of whether regularization can improve generalization.

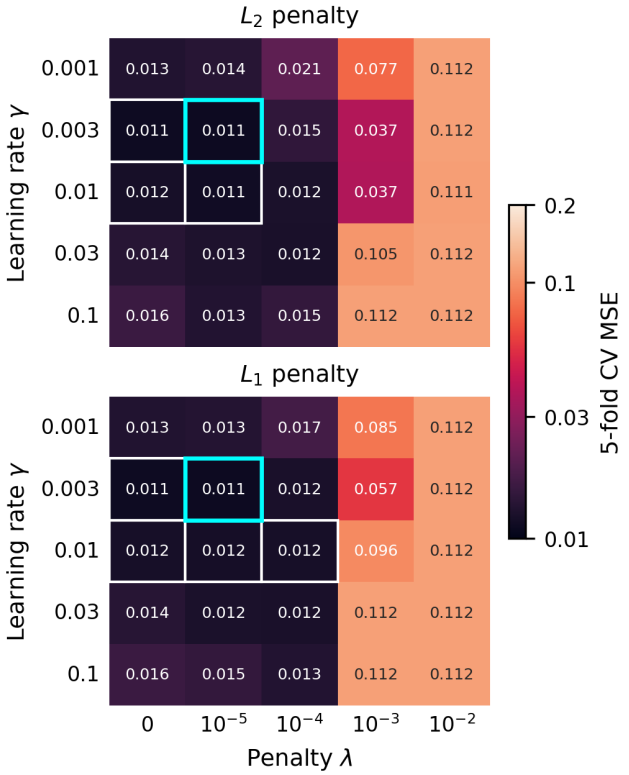


FIG. 6. Five-fold cross-validation MSE over the learning rate γ and the penalty λ for the L_2 (top) and L_1 (bottom) penalties, network [25, 25] (sigmoid). White boxes: within one standard error of the minimum. The colored box marks the minimum.

Table III compares test MSE for Ridge, Lasso and the networks with L_2 and L_1 penalties. Both networks use $\lambda = 10^{-5}$, selected by cross-validation, and give almost identical test MSEs of 0.0073 and 0.0074. The linear models differ more: Ridge gives 0.0061, while Lasso gives 0.0093. However, with 20 test points, SD $\approx 0.32 \cdot$ MSE, giving approximately 0.0019 for Ridge, 0.0023 for the networks and 0.0030 for Lasso. As in Section IV B 1, the differences are small compared with this uncertainty. The networks therefore show no clear improvement over Ridge, while Lasso's higher error is a tendency rather than a firm conclusion.

With $\lambda = 6.9 \times 10^{-9}$, Ridge is effectively OLS, giving the same MSE as in Section IV A. This is consistent with our earlier findings: the Runge function is smooth and one-dimensional, and a degree-12 polynomial already fits it well with 80 training points. The network has around 700 parameters and is trained with gradient-based optimization, but its extra flexibility gives no clear prediction gain here.

TABLE III. Test MSE for Ridge and Lasso regression (degree 12, λ from project 1) and for the network [25, 25] (sigmoid) with the L_2 and L_1 penalty. For the networks, λ minimizes the 5-fold cross-validation MSE (Fig. 6), with $\gamma = 3 \cdot 10^{-3}$.

Model	λ	Test MSE
Ridge	$6.9 \cdot 10^{-9}$	**0.0061**
NN, L_2	10^{-5}	0.0073
Lasso	$2.9 \cdot 10^{-5}$	0.0093
NN, L_1	10^{-5}	**0.0074**

To see whether the similar prediction errors hide differences in the weights, we trained the network [25, 25] with the same data and seed, without a penalty, with L_2 and with L_1 ($\lambda = 10^{-5}$, $\gamma = 3 \cdot 10^{-3}$). Figure 7 shows the absolute values of all 675 weights sorted from largest to smallest. The L_2 penalty leaves the weights almost unchanged, while L_1 pushes 27% below 10^{-3} and leaves the largest weights about the same size.

This follows from the penalty gradients in Eq. (6): L_1 pulls each nonzero weight towards zero with magnitude λ, while the L_2 contribution is $2\lambda w$ and becomes weaker as the weight decreases. This is the same mechanism behind the different effects of Lasso and Ridge. That many weights can become nearly zero with almost unchanged test MSE suggests that the network has redundant weights, consistent with its extra flexibility giving little benefit for this problem.

For this smooth, one-dimensional problem, the network shows no clear advantage over a well-chosen linear model; we next examine whether its flexibility is more useful for the high-dimensional Fashion-MNIST data.

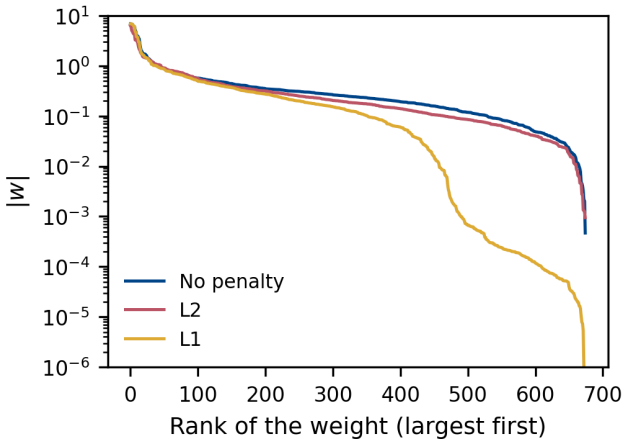


FIG. 7. Absolute values of the trained weights of the network [25, 25], sorted from largest to smallest, with no penalty, L_2 and L_1 ($\gamma = 3 \cdot 10^{-3}$, $\lambda = 10^{-5}$). The share of weights with $|w| < 10^{-3}$ is 0.00, 0.00 and 0.27.

C. Classification of Fashion-MNIST

Now, we are exploring how a network performs in classification using the Fashion-MNIST data set as described in Section III B.

1. *Baseline and hyperparameters*

We start with logistic regression, our network without hidden layers, as a baseline. Table IV shows the highest validation accuracy of 0.852 at $\gamma = 10^{-3}$, compared with 0.10 for random guessing. The learning rate must suit the fixed budget of 10 epochs: a smaller rate learns slowly, while a larger rate can make training unstable. Training accuracy of 0.860 is close to validation accuracy, suggesting little overfitting. A hidden layer must therefore improve on a reasonably well-generalizing linear classifier.

TABLE IV. Logistic regression (our network with no hidden layers, softmax output, cross entropy), Adam, 10 epochs, batch size 100. Accuracy on the training data and on the validation set (20 % of the 60 000 training images), and the training cost after the last epoch.

γ	Train acc.	Val. acc.	Final cost
10^{-4}	0.826	0.820	0.529
10^{-3}	0.860	**0.852**	0.406
10^{-2}	0.859	0.841	0.402
10^{-1}	0.804	0.793	1.941

We next search jointly over the learning rate γ and the L_2 penalty λ for a network with 100 sigmoid nodes. Figure 8 gives the highest validation accuracy of 0.889 at $\gamma \approx 3.2 \times 10^{-3}$ and $\lambda = 10^{-6}$. The best penalty is

the smallest tested, so the search gives no evidence that stronger regularization helps; since $\lambda = 0$ was not tested, we cannot determine whether a penalty is needed. At $\lambda = 10^{-1}$, accuracy falls to 0.23–0.32, indicating underfitting as the penalty restricts the network's ability to learn.

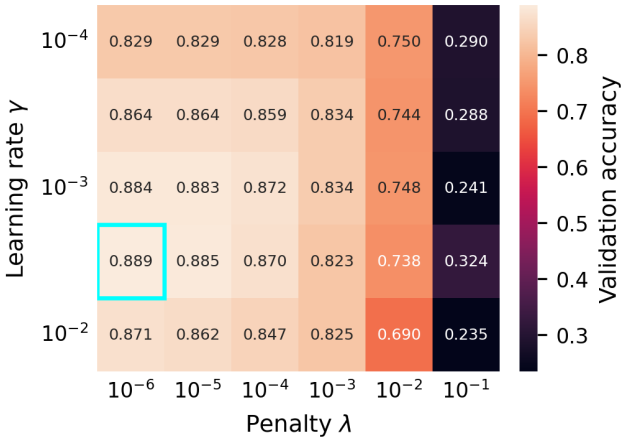


FIG. 8. Validation accuracy of the network [100] (sigmoid, L_2) for five learning rates γ, evenly spaced on a log scale, and six penalties λ. Box: the highest accuracy.

We then compare architectures using $\lambda = 10^{-6}$ and $\gamma = 3 \times 10^{-3}$, rounded from the grid-search value. Figure 9 shows that all tested networks outperform logistic regression. Increasing width generally helps, while adding layers gives little improvement. The best validation accuracies are similar: 0.896 for sigmoid, 0.894 for ReLU and 0.892 for Leaky ReLU. With 12 000 validation images, the binomial estimate $\mathrm{SD}(\widehat{\mathrm{acc}}) = \sqrt{\widehat{\mathrm{acc}}(1-\widehat{\mathrm{acc}})/n}$ is approximately 0.003, so these small differences do not establish a clear ranking.

Sigmoid [200] and [200, 200] also perform almost identically, at 0.896 and 0.895. We choose [200] because it achieves similar accuracy with fewer parameters. Unlike for Runge, the activations show similar stability across architectures. However, the comparison uses a learning rate and penalty selected for sigmoid [100], and other architectures may benefit from different settings or more training.

2. *Final models on the test set*

After selecting the hyperparameters, we retrain the final models on all 60 000 training images and evaluate them on the 10 000 test images. Table V shows test accuracy of 0.875 for our network and 0.844 for logistic regression, an improvement of approximately 3 percentage points. The sampling SD is about 0.003–0.004 for each accuracy, making this a much clearer improvement than the differences between the best architectures. Unlike for Runge, a nonlinear hidden layer provides a useful

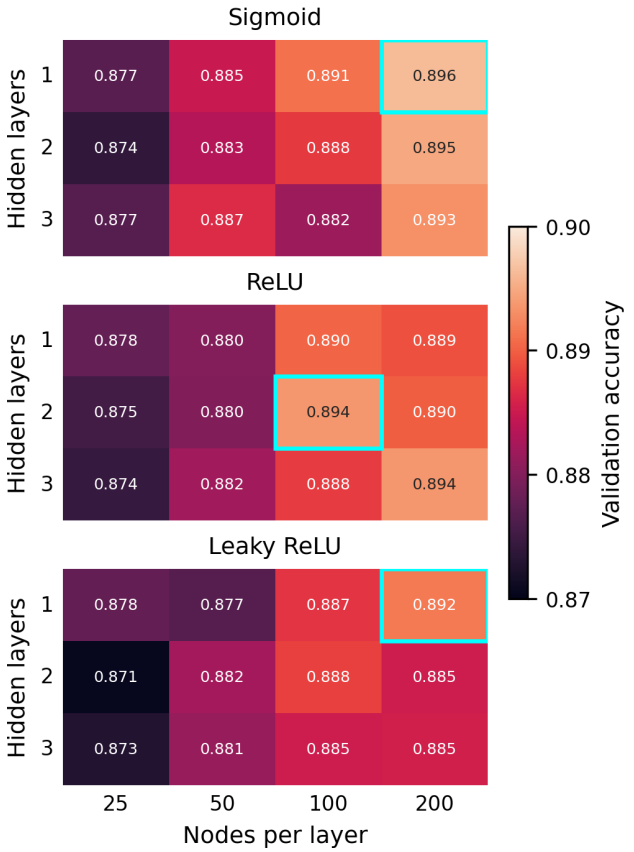


FIG. 9. Validation accuracy for sigmoid, ReLU and Leaky ReLU hidden layers as a function of the number of hidden layers and nodes per layer ($\gamma = 3 \cdot 10^{-3}$, $\lambda = 10^{-6}$). Box: the highest accuracy for each activation.

gain here, at the cost of about 20 times as many parameters and twice the training time.

Scikit-Learn's logistic regression reaches the same accuracy as ours, suggesting that the linear model itself limits performance. Its longer runtime is not directly comparable because the optimization methods and stopping criteria differ. `MLPClassifier` reaches 0.883, placing our implementation in a similar accuracy range. The Fashion-MNIST benchmark reports up to 0.842 for logistic regression and 0.877 for a ReLU network with 100 hidden nodes [13], matching our own results. Our network's test accuracy is lower than its validation accuracy of 0.896. Selecting the best of many configurations can make validation results optimistic, while retraining introduces additional variation; these runs cannot separate the contributions.

Figure 10 shows recall on the diagonal, with errors concentrated among visually similar classes. For example, 20% of T-shirts are classified as Shirt, while Coat and Pullover are also frequently confused. Similar silhouettes make these classes harder to separate than Trouser and Bag. Figure 11 illustrates individual errors, including some with high predicted probabilities. These examples

TABLE V. Test accuracy on the 10 000 Fashion-MNIST test images for the selected models, with the number of parameters and the training time. LR is logistic regression and NN the network with one hidden layer of 200 sigmoid nodes; the Scikit-Learn network is `MLPClassifier`. Guessing gives 0.10.

Model	Parameters	Test acc.	Time (s)
LR (ours)	7 850	0.844	19
LR (Scikit-Learn)	7 850	0.844	632
NN [200] (ours)	159 010	0.875	38
NN [200] (Scikit-Learn)	159 010	**0.883**	38

show that a high predicted probability does not guarantee a correct classification.

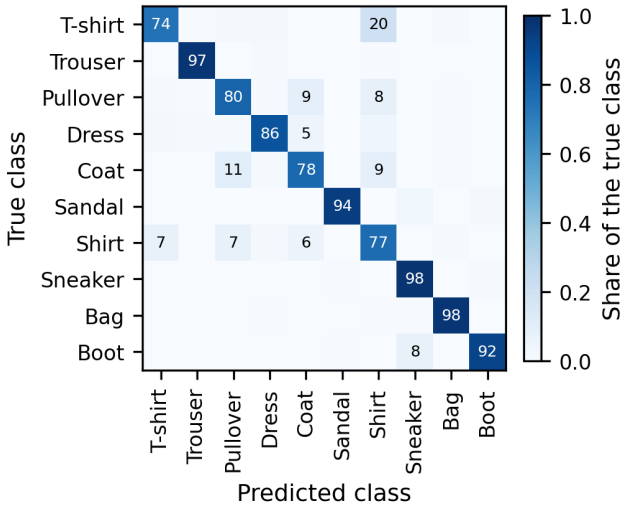


FIG. 10. Confusion matrix of the network [200] (sigmoid) on the test set, as the percentage of each true class. Numbers are shown on the diagonal and for confusions of 5% or more.

Table VI shows where the overall improvement comes from. Since the test set has 1000 images per class, accuracy is the mean class recall. Shirt recall rises from 0.52 to 0.77, contributing about 2.5 percentage points, or roughly 80% of the net accuracy gain. However, the network also predicts Shirt more often, and T-shirt recall falls from 0.82 to 0.74. Its unchanged F_1 of 0.81 reflects an increase in precision that offsets this loss in recall. The network therefore changes the balance between similar classes, while Shirt remains the weakest class by F_1. Dress and Ankle boot also have lower recall than with logistic regression.

Figure 12 shows lower training cross-entropy for the network throughout the 10 epochs, consistent with its greater capacity. Neither curve has fully flattened, so the fixed training budget is a limitation. Training loss alone cannot tell whether more epochs would improve test accuracy or lead to overfitting.

Overall, the hidden layer gives a clear improvement over logistic regression on Fashion-MNIST, although

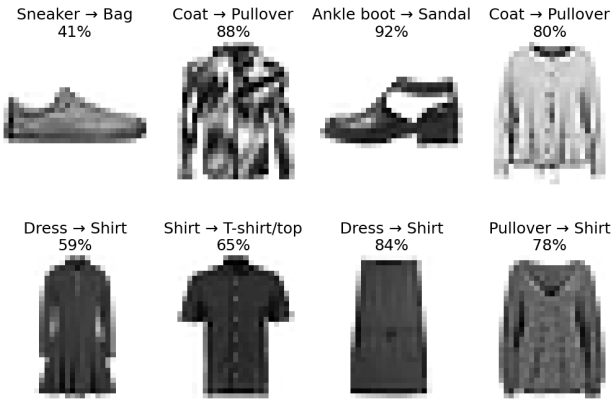


FIG. 11. The first eight of the 1 248 misclassified test images, with the true and the predicted class and the predicted probability of the network [200].

TABLE VI. Recall and F_1 score per class on the test set for logistic regression (LR) and the network [200] (NN).

	Recall		F_1	
Class	LR	NN	LR	NN
T-shirt/top	0.82	0.74	0.81	0.81
Trouser	0.95	0.97	0.97	0.98
Pullover	0.74	0.80	0.73	0.80
Dress	0.88	0.86	0.85	0.89
Coat	0.77	0.79	0.74	0.79
Sandal	0.92	0.94	0.93	0.96
Shirt	0.52	0.77	0.58	0.70
Sneaker	0.92	0.98	0.92	0.94
Bag	0.95	0.98	0.95	0.97
Ankle boot	0.96	0.92	0.94	0.94
Accuracy			0.84	0.88

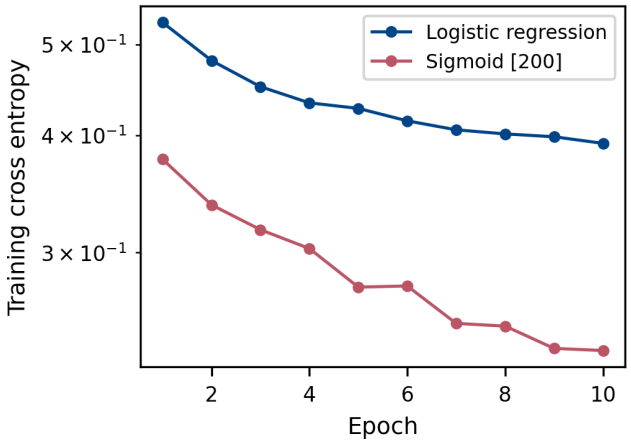


FIG. 12. Training cross entropy per epoch for logistic regression ($\gamma = 10^{-3}$) and the network [200] ($\gamma = 3 \cdot 10^{-3}$, $\lambda = 10^{-6}$).

much of the gain comes from better recognition of Shirt and confusion between similar classes remains.

V. CONCLUSION

A. Final words

So when is a neural network worth it? On high-dimensional data with lots of structure it is worth the extra model complexity, but not necessarily on a one-dimensional smooth problem like Runge.

On the Runge data, the network matches OLS, but does not outperform it with an MSE of 0.0065 against 0.0061. The prediction error arises in different parts for the two models. OLS oscillates near the end points, while the network has smooth edges but overshoots the top of the function.

For Fashion-MNIST the network beats logistic regression in test accuracy by 3.1 percentage points, a real improvement, but it costs 20 times as many parameters. The biggest gains come from non-linear details in some of the classes. Our own implementation of a network comes close to `MLPClassifier`, which got 0.883 test accuracy with the same run time.

Adam is the most robust optimizer across learning rates. For activation, stability across architectures mattered more than the lowest MSE. Sigmoid is the most stable across different architectures. Regularization can only help when the model overfits. The network [25, 25] barely overfitted, so L_2 and L_1 gave no measurable advantage.

With that said, our biggest limiting factors are only testing on one seed, and only using 100 data points. With an 80/20 split, our test set is restricted to only 20 points. This results in a standard deviation of $0.32 \cdot \text{MSE} \approx 0.002$–$0.003$ (Sec. IV B 2), which is of the same magnitude as the differences between our models. Hence, they cannot be ranked. Our training set also suffers from the same restriction. Our models are trained on only 80 data points and sigmoid [100, 100] has 10 401 parameters. With so little data, the results strongly depend on the seed. The seed could also be the reason why all our test MSE lie near $\sigma^2 = 0.01$. Our 20 test points probably had very little noise, which could be different for other seeds.

B. Future work

In our Fashion-MNIST data, each image is flattened to a vector, so the network does not know that pixel (i, j) lies next to $(i, j + 1)$. Reshuffling the pixels in the same way for all images would not change the outcome of the training. In contrast to our feed-forward network, a convolutional network would respect the structure of the data [11, p. 399]. It would therefore probably reach a higher accuracy, especially for the class "Shirt", which is often confused with "T-shirt" and "Pullover". By separating local details, the convolutional network could distinguish the classes by e.g. buttons or collars.

[1] Y. LeCun, Y. Bengio, and G. Hinton, Deep learning, Nature **521**, 436 (2015).

[2] G. Cybenko, Approximation by superpositions of a sigmoidal function, Mathematics of Control, Signals and Systems **2**, 303 (1989).

[3] C. Runge, Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten, Zeitschrift für Mathematik und Physik **46**, 224 (1901).

[4] A. Sjåvåg, T. Aursnes, R. Rauzy, and J. S. Florholmen, *Regression Analysis, Resampling Methods and Gradient Descent*, FYS-STK3155 Project 1 report (Department of Physics, University of Oslo, 2026) unpublished. Code available at `https://github.com/rkrauzy/Project1_FYSSTK3155`.

[5] A. E. Hoerl and R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics **12**, 55 (1970).

[6] R. Tibshirani, Regression shrinkage and selection via the lasso, Journal of the Royal Statistical Society: Series B **58**, 267 (1996).

[7] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018).

[8] H. Xiao, K. Rasul, and R. Vollgraf, Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms (2017), arXiv:1708.07747 [cs.LG].

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research **12**, 2825 (2011).

[10] I. Neutelings, Neural networks, `https://tikz.net/neural_networks/` (2024), tikZ.net, licensed under CC BY-SA 4.0. Accessed 27 October 2026.

[11] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning* (Springer, 2026) lecture notes for FYS-STK3155/4155, University of Oslo, version of September 23, 2026.

[12] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in *International Conference on Learning Representations (ICLR)* (2015) arXiv:1412.6980.

[13] Zalando Research, Fashion-MNIST benchmark dashboard, `http://fashion-mnist.s3-website.eu-central-1.amazonaws.com`, accessed 9 November 2026.

APPENDIX

Appendix A: Additional derivations

1. Derivative of the sigmoid function

The expression for $\sigma'(z)$ used in Section II A 2 follows from

$$\begin{aligned}\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\ &= \frac{1}{1+e^{-z}} \cdot \frac{(1+e^{-z})-1}{1+e^{-z}} \\ &= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right) \\ &= \sigma(z)\big[1-\sigma(z)\big].\end{aligned} \tag{A1}$$

2. Gradient of the binary cross-entropy

Differentiating Equation (2) with respect to the output gives

$$\begin{aligned}\frac{\partial C_{\mathrm{BCE}}}{\partial a_i^L} &= \frac{1}{n}\left[-\frac{y_i}{a_i^L} + \frac{1-y_i}{1-a_i^L}\right] \\ &= \frac{1}{n}\frac{a_i^L - y_i}{a_i^L\left(1-a_i^L\right)}.\end{aligned} \tag{A2}$$

Using the sigmoid derivative from Equation (1), the chain rule gives

$$\begin{aligned}\frac{\partial C_{\mathrm{BCE}}}{\partial z_i^L} &= \frac{\partial C_{\mathrm{BCE}}}{\partial a_i^L}\, a_i^L\left(1-a_i^L\right) \\ &= \frac{1}{n}\left(a_i^L - y_i\right).\end{aligned} \tag{A3}$$

3. Gradient of softmax with cross-entropy

Each softmax output in Equation (3) depends on all scores for the same observation. The quotient rule gives

$$\frac{\partial a_{ik}^L}{\partial z_{ij}^L} = a_{ik}^L\left(\mathbf{1}_{\{k=j\}} - a_{ij}^L\right), \tag{A4}$$

where $\mathbf{1}_{\{k=j\}}$ equals 1 when $k=j$ and 0 otherwise. Writing the loss for one observation as $C_i = -\sum_k y_{ik}\log a_{ik}^L$, the chain rule gives

$$\begin{aligned}\frac{\partial C_i}{\partial z_{ij}^L} &= -\sum_{k=0}^{K-1}\frac{y_{ik}}{a_{ik}^L}\frac{\partial a_{ik}^L}{\partial z_{ij}^L} \\ &= -\sum_{k=0}^{K-1} y_{ik}\left(\mathbf{1}_{\{k=j\}} - a_{ij}^L\right) \\ &= -y_{ij} + a_{ij}^L\sum_{k=0}^{K-1} y_{ik} \\ &= a_{ij}^L - y_{ij}.\end{aligned} \tag{A5}$$

The last equality follows from $\sum_k y_{ik} = 1$. Averaging over observations gives Equation (4).

4. Sampling uncertainty of the test MSE

The test MSE is estimated from m test points,

$$\widehat{\mathrm{MSE}} = \frac{1}{m}\sum_{i=1}^{m} e_i^2, \qquad e_i = y_i - \hat{f}(x_i). \tag{A6}$$

To estimate how much $\widehat{\mathrm{MSE}}$ would vary with a different test set, we make two assumptions:

1. The model $\hat{f}$ is fixed after training and the test points are drawn independently, so the errors $e_1, \ldots, e_m$ are independent.
2. The errors are approximately normally distributed, $e_i \sim N(0, s^2)$. This is reasonable here because the noise is Gaussian by construction and all test errors lie close to σ^2, so the model error is small compared with the noise and $e_i \approx \varepsilon_i$.

Under assumption 2, $e_i^2/s^2 \sim \chi_1^2$, which has mean 1 and variance 2. Hence

$$\mathbb{E}[e_i^2] = s^2, \qquad \mathrm{Var}(e_i^2) = 2s^4. \tag{A7}$$

Using assumption 1, the variance of the average is

$$\mathrm{Var}\big(\widehat{\mathrm{MSE}}\big) = \frac{1}{m^2}\sum_{i=1}^{m}\mathrm{Var}(e_i^2) = \frac{2s^4}{m}, \tag{A8}$$

so the standard deviation is $s^2\sqrt{2/m}$. Replacing the unknown s^2 by the measured MSE gives

$$\mathrm{SD}\big(\widehat{\mathrm{MSE}}\big) \approx \widehat{\mathrm{MSE}}\,\sqrt{2/m}. \tag{A9}$$

With $m = 20$ test points, $\sqrt{2/20} \approx 0.32$. For the values in Table I this gives SD ≈ 0.0019 for OLS ($\widehat{\mathrm{MSE}} = 0.0061$) and SD ≈ 0.0032 for the network [100] ($\widehat{\mathrm{MSE}} = 0.0100$).

This estimate only accounts for the randomness of the test set. Randomness from training (initialisation and minibatch order) is not included, since $\hat{f}$ is treated as fixed. If the errors are not centred, the variance increases: for $e_i \sim N(\mu, s^2)$ one finds $\mathrm{Var}(e_i^2) = 2s^4 + 4\mu^2 s^2$. The estimate above should therefore be read as a rough lower bound on the uncertainty of a single MSE value.

Appendix B: Declaration of AI use

1. AI tools used

- Claude Code (Anthropic, October–November 2026)

- Claude Sonnet 5.5 via claude.ai (Anthropic, October–November 2026), used for feedback on text, structure, LaTeX, and for checking numbers in the text against the output files.

2. Text writing and editing

The text contribution levels are: Level 0 (none), Level 1 (language only: spell-check, grammar and minor phrasing) and level 2 (Editorial: LLM rewrote or restructured sentences/paragraphs; we supplied all ideas and checked the result) . All text in the report was written by the authors.

3. Code generation and assistance

The code contribution levels are: Level 0 (none), Level 1 (debugging), Level 2 (snippet), Level 3 (skeleton), and Level 4 (substantial).

All submitted code was reviewed, tested and understood by the project authors.

TABLE VII. LLM assistance for the report text (Claude Sonnet 5.5, claude.ai).

Section	Level	Description
Abstract	1	Claude marked language and grammar errors. Text written by us.
Introduction	1	Claude marked language and grammar errors. Text written by us.
Theory	1	Claude marked language and LaTeX errors, aligned our notation with the lecture notes, and commented on the order of the subsections and what each should cover. All text, derivations and equations written by us.
Implementation	2	Claude suggested the outline, pointed out errors and repetitions, and proposed rewording.
Results and discussion: Runge function	1	Claude marked language, grammar and LaTeX errors, and checked that every number in the text matches the output files in the repository. Text written by us.
Results and discussion: tuning and regularization	1	Claude marked language, grammar and LaTeX errors and broken references, and checked the numbers against the output files. Text written by us.
Results and discussion: Fashion-MNIST	1	Claude marked language, grammar and LaTeX errors, and checked the numbers against the output files. Text written by us.
Conclusion	1	Claude marked language errors and statements that did not match the results sections. Text written by us.
Appendix	1	Claude added subsection titles, labels and cross-references. Derivations written by us.

TABLE VIII. LLM assistance for shared source files and tests.

File	Level	Description
`src/activations.py`	0	Code from our week 43 exercises and Section 8.13 of the lecture notes. `leaky_relu` was taken from the lecture notes.
`src/costs.py`	0	Copied from our week 42 exercises (`mse` from the template, `mse_der` our own, `cross_entropy` from the lecture notebook via LLM help in week 42).
`src/optimiser.py`	0	`optimiser_step` and `make_batches` unchanged from project 1.
`src/data.py`	0	Runge data from project 1. `load_fashion_mnist` is our earlier `load_mnist` from the week 43 lecture notes, with one line changed to load Fashion-MNIST.
`src/plotting.py`	0	No contribution from Claude in this project.
`src/models.py`	1	`ols`, `ridge` and `lasso_fit` from project 1. Claude found that `lasso_fit` did not converge at degree 12 and raised `max_iter` from 10^5 to 10^7.
`src/neural_network.py`	3	Written by us from the week 42 template, with close guidance. Claude showed how our week 42 functions translate to methods (`predict`, `_feed_forward_saver`, `cost`, `compute_gradient`), combined the softmax shortcut and the L_1/L_2 terms in `compute_gradient`, gave a skeleton for `_create_layers`, proposed `update_weights`, `train` (from our project 1 `sgd`) and the two JAX methods, and helped find bugs. Claude added He initialisation for ReLU and Leaky ReLU, following `create_layers` in our week 43 exercises.
`tests/test_implementation.py`	4	Test functions proposed and written by Claude: shape checks, backpropagation against `jax.grad`, training lowers the cost, rejection of 1D targets and the L_2 penalty against `jax.grad`. Claude wrote `test_softmax_matches_jax`, following the softmax check in our week 42 exercises (Exercise 7, iris).
`run_all.ipynb`	3	Written by Claude on our request, after the project 1 feedback asked for one way to run everything. Each cell runs one of our scripts, saves the output in `outputs/` and shows its figure.

TABLE IX. LLM assistance for Part B.

File	Level	Description
`parts/part_b/nn_vs_ols.py`	2	Data and the first network written by us. Claude found that the OLS fit lacked the scaling and centring from project 1 and added it, following our project 1 `ridge_baseline.py` (itself LLM-assisted, level 2), and wrote the loop over the four architectures.
`parts/part_b/learning_rate.py`	3	First version written by Claude, built on our `nn_vs_ols.py` and the loop structure of our project 1 `sgd_methods.py`. Claude wrote the plot of validation MSE against the learning rate.
`parts/part_b/fit.py`	2	Data, OLS and network written by us from our `nn_vs_ols.py`. Claude suggested the decomposition of the network into the terms $v_j\sigma(w_j x + b_j)$ (two lines), fixed bugs (OLS refitted on the grid, 1D targets), and wrote the plotting of the two panels.

TABLE X. LLM assistance for Parts C and D.

File	Level	Description
`parts/part_c/activation_depth.py`	2	Loops over activation, layers and nodes written by us, from our `nn_vs_ols.py`. Claude added the validation split, the grid of validation MSE, the printout of training and validation MSE and the Seaborn heat maps.
`parts/part_d/cv_norms.py`	2	Assembled by Claude from our week 44 exercises (the Scikit-Learn wrapper around our network, `GridSearchCV`, standard errors and the one-standard-error rule) and our earlier `norms.py` (heat map, comparison with Ridge and Lasso). Replaces `norms.py`, which used a single validation split.
`parts/part_d/weights.py`	2	Training loop from our `norms.py` and plotting from panel (c) of the penalty figure in the week 41 Monday lecture. Claude wrote the list of runs, the line collecting the weights of all layers, and the printout of the share of weights below 10^{-3}.

TABLE XI. LLM assistance for Part E.

File	Level	Description
`parts/part_e/logreg.py`	2	Written by us from our `learning_rate.py` and week 42 exercises. Claude explained the accuracy line, suggested 10 epochs and batch size 100 (from our week 43 exercises), showed how to track the best γ with our `norms.py` pattern, and found bugs (training twice, wrong comparison sign).
`parts/part_e/hyperparameters.py`	2	Copied by us from our `norms.py`. Claude converted the remaining regression parts to classification (network, one-hot targets, accuracy, best accuracy, heat map labels).
`parts/part_e/architecture.py`	1	Copied by us from our `activation_depth.py` and adapted to classification. Claude found bugs (missing `cost_der`, 10 instead of 784 inputs, best activation not stored, number format in the printout).
`parts/part_e/best_model.py`	2	Written by us. Claude found bugs (logistic regression given a sigmoid output layer, a missing derivative, overwritten predictions), wrote the loop printing test accuracy and number of parameters and the confusion matrix figure, and pointed to Section 5.13 of the lecture notes for `confusion_matrix` and `classification_report`.
`parts/part_e/examples.py`	1	Written by us from the MNIST images in the week 41 Monday lecture. Claude suggested `np.where` to find one image per class and `ax.axis("off")`.
`parts/part_e/learning_curves.py`	2	Data and network from our `best_model.py`. Claude wrote the training of both models with the returned cost history and the `semilogy` plot, following the week 43 Tuesday notebook.
`parts/part_e/sklearn_compare.py`	1	Written by us from the `LogisticRegression` example in the assignment and the `MLPClassifier` example in Section 8.14 of the lecture notes. Claude found bugs (combined timing, swapped labels, training instead of test accuracy).